420-301-VA: Web Services

Lab 6: Middleware, Pagination, and Input Validation

Pravish Sainath
Computer Science & Technology
Vanier College

Introduction

In Lab 5, you built a working CRUD API using PHP and PDO. While functional, a real-world web service requires more than basic database queries to survive in production. What happens if your database table grows to 100,000 records and a client calls `GET /books`? The server will crash trying to load everything into memory. What happens if a client sends malformed data, or requests XML instead of JSON?

In this lab, you will upgrade your API (and prepare for your Team Project deliverables) by implementing essential production features:

- **Middleware Pipeline & Content Negotiation:** Intercepting requests before they hit your routing logic to enforce headers (like `Accept` and `Content-Type`) and handle cross-cutting concerns.
- **Filtered & Paginated Collections:** Using URL query parameters (`$_GET`) alongside SQL `LIMIT`, `OFFSET`, and `WHERE` clauses to return manageable chunks of data wrapped in a metadata envelope.
- **Input Validation & Structured Error Handling:** Sanitizing and validating incoming JSON payloads, and returning standardized, descriptive error responses (e.g., `400 Bad Request`, `406 Not Acceptable`, `415 Unsupported Media Type`, `422 Unprocessable Entity`).

Objectives

By the end of this lab, you should be able to:

- Understand the concept of an HTTP Middleware Pipeline.
- Implement Content Negotiation by inspecting the `Accept` and `Content-Type` headers.
- Configure a global exception handler in PHP to ensure the API never leaks raw HTML stack traces, always returning structured JSON errors.

- Implement dynamic SQL queries using PDO to support filtering and pagination via query parameters.
- Build a robust input validation layer for `POST` and `PUT` requests.
- Apply these architectural patterns directly to your Team Project repository.

Instructions

- You will continue modifying the `library_db` and `api.php` project you started in Lab 5.
- Use Postman or Insomnia to test all endpoints, specifically manipulating the **Headers** and **Params** tabs to trigger your new logic.
- In Part 4, you will transition to your Team Project repository to implement these exact patterns for your group's core resources.
- Document your code and capture screenshots of your Postman tests (especially the error states and paginated responses) for your lab submission.

1 Part 1: Middleware, Content Negotiation, and Error Handling

1.1 Concept Review: The Middleware Pipeline

In modern web frameworks (like Slim, Express, or Spring Boot), an HTTP request passes through a series of layers—called **Middleware**—before it ever reaches your controller or database logic. Think of middleware as security checkpoints at an airport. Each layer inspects the request: if something is wrong, the middleware immediately halts execution and returns an error response. If everything passes, the request moves to the next layer.

Common middleware responsibilities include:

1. **Error Handling:** Catching unexpected crashes and formatting them cleanly.
2. **Content Negotiation:** Ensuring the client and server agree on the data format (JSON).
3. **Authentication / Rate Limiting:** (Covered in future weeks).

Exercise 1: Building the Pipeline Checkpoints

Step 1A: Global JSON Error Handling

By default, if PHP encounters a fatal error or uncaught exception, it prints an HTML error message. If a client app is expecting JSON, parsing this HTML will cause the client app to crash. We can fix this by registering a global exception handler at the very top of `api.php`.

Add the following helper function and exception handler right after your `<?php` tag:

```php
<?php
// Helper function to send standardized JSON error responses
function sendError(int $statusCode, string $message, array $details = []) {
    http_response_code($statusCode);
    $response = [
        "status" => $statusCode,
        "error" => $message
    ];
    if (!empty($details)) {
        $response["details"] = $details;
    }
    echo json_encode($response);
    exit; // Stop execution immediately
}

// Global Exception Handler Middleware
set_exception_handler(function ($exception) {
    // In production, you would log $exception->getMessage() to a file
    // rather than exposing internal database details to the user.
    sendError(500, "Internal Server Error", [
        "debug_message" => $exception->getMessage()
    ]);
});
```

Step 1B: Content Negotiation Middleware

Content negotiation is the mechanism where the client tells the server what format it wants to receive (via the `Accept` header) and what format it is sending (via the `Content-Type` header). Since our API is strictly a JSON service, we must reject requests that demand XML or send plain text.

Add this middleware function right below your exception handler, and call it before your routing logic:

```
function runContentNegotiationMiddleware(string $method) {
    // 1. Check the 'Accept' header (What the client wants back)
    $acceptHeader = $_SERVER['HTTP_ACCEPT'] ?? '*/*';

    // If client doesn't accept JSON or wildcard (*/*), reject with 406
    if (strpos($acceptHeader, 'application/json') === false &&
        strpos($acceptHeader, '*/*') === false) {
        sendError(406, "Not Acceptable. This API only provides application/json.");
    }

    // 2. Check 'Content-Type' for methods that send a body
    if (in_array($method, ['POST', 'PUT', 'PATCH'])) {
        $contentType = $_SERVER['CONTENT_TYPE'] ?? '';

        // If they aren't sending JSON, reject with 415
        if (strpos($contentType, 'application/json') === false) {
            sendError(415, "Unsupported Media Type. Please send application/json.");
        }
    }
}

// --- Execute Pipeline ---
header("Access-Control-Allow-Origin: *");
header("Content-Type: application/json; charset=UTF-8");
header("Access-Control-Allow-Methods: GET, POST, PUT, PATCH, DELETE");

$method = $_SERVER['REQUEST_METHOD'];
$uri = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);

// Run the middleware!
runContentNegotiationMiddleware($method);
```

Step 1C: Test the Middleware in Postman

1. **Test 406 Not Acceptable:** In Postman, set up a `GET` request to `http://localhost/api.php/books`. Go to the **Headers** tab. Uncheck the default `Accept` header and add a custom one: Key: `Accept`, Value: `application/xml`. Click Send. Verify you receive a `406` status code with your structured JSON error!

2. **Test 415 Unsupported Media Type:** Change the method to `POST`. Go to the **Body** tab, select `raw`, and change the dropdown from `JSON` to `Text`. Send any text. Verify you receive a `415` status code!

2 Part 2: Paginated and Filtered Collection Responses

2.1 Concept Review: Handling Large Datasets

When a client requests a collection (`GET /books`), returning every row in the database is dangerous. Instead, we use **Pagination** to slice the data into "pages" using SQL's `LIMIT` and `OFFSET` clauses.

- `limit` (or `per_page`): How many items to return (e.g., 5).
- `page`: Which page number the user wants (e.g., page 2).
- The math to calculate the SQL offset is: $\text{offset} = (\text{page} - 1) \times \text{limit}$.

Furthermore, when returning paginated data, we should wrap the array in a JSON object (an "envelope") that includes **metadata** so the frontend knows how many total pages exist.

Exercise 2: Upgrading the GET Collection Endpoint

Step 2A: Seed More Data

To properly test pagination and filtering, you need more than two books in your database. Run the following SQL in phpMyAdmin on your `library_db`:

```sql
INSERT INTO books (title, author, is_available) VALUES
('The Hobbit', 'J.R.R. Tolkien', 1),
('The Fellowship of the Ring', 'J.R.R. Tolkien', 0),
('The Two Towers', 'J.R.R. Tolkien', 1),
('The Return of the King', 'J.R.R. Tolkien', 1),
('Foundation', 'Isaac Asimov', 1),
('I, Robot', 'Isaac Asimov', 0),
('Brave New World', 'Aldous Huxley', 1),
('Fahrenheit 451', 'Ray Bradbury', 1);
```

Step 2B: Implementing Filtering and Pagination

Replace your existing `if ($method === 'GET')` block inside `/api.php/books` with the following advanced query builder. Notice how we dynamically build the `WHERE` clause and bind parameters safely!

```php
if ($method === 'GET') {
    // 1. Extract Pagination Params (with sensible defaults)
    $page = isset($_GET['page']) ? (int)$_GET['page'] : 1;
    $limit = isset($_GET['limit']) ? (int)$_GET['limit'] : 5;

    // Enforce boundaries (prevent negative numbers or massive limits)
    if ($page < 1) $page = 1;
    if ($limit < 1) $limit = 5;
    if ($limit > 50) $limit = 50; // Max 50 items per page

    $offset = ($page - 1) * $limit;

    // 2. Build Dynamic Filtering (WHERE clauses)
    $whereClauses = [];
    $queryParams = [];
```

```php
// Filter by Author (exact or partial match)
if (!empty($_GET['author'])) {
    $whereClauses[] = "author LIKE :author";
    $queryParams['author'] = "%" . $_GET['author'] . "%";
}

// Filter by Availability (0 or 1)
if (isset($_GET['is_available']) && $_GET['is_available'] !== '') {
    $whereClauses[] = "is_available = :is_available";
    $queryParams['is_available'] = (int)$_GET['is_available'];
}

// Combine WHERE clauses if any exist
$whereSql = "";
if (count($whereClauses) > 0) {
    $whereSql = "WHERE " . implode(" AND ", $whereClauses);
}

// 3. Get Total Count (for metadata)
$countSql = "SELECT COUNT(*) FROM books $whereSql";
$countStmt = $pdo->prepare($countSql);
$countStmt->execute($queryParams);
$totalItems = (int)$countStmt->fetchColumn();
$totalPages = ceil($totalItems / $limit);

// 4. Fetch the Paginated Data
$dataSql = "SELECT * FROM books $whereSql LIMIT :limit OFFSET :offset";
$dataStmt = $pdo->prepare($dataSql);

// Bind filter params
foreach ($queryParams as $key => $val) {
    $dataStmt->bindValue(":$key", $val);
}
// Bind pagination params explicitly as integers!
$dataStmt->bindValue(':limit', $limit, PDO::PARAM_INT);
$dataStmt->bindValue(':offset', $offset, PDO::PARAM_INT);

$dataStmt->execute();
$books = $dataStmt->fetchAll();

// 5. Return Enveloped Response
http_response_code(200);
echo json_encode([
    "metadata" => [
        "total_items" => $totalItems,
        "total_pages" => $totalPages,
        "current_page" => $page,
        "per_page" => $limit
    ],
    "data" => $books
]);
```

```
}
```

Step 2C: Test Your Queries in Postman

- Request `GET http://localhost/api.php/books?page=1&limit=3`. Observe the `metadata` block and verify only 3 books are returned.
- Request `GET http://localhost/api.php/books?author=Tolkien&is_available=1`. Verify that only available books by Tolkien are returned, and that `total_items` reflects the filtered count!

3 Part 3: Input Validation

3.1 Concept Review: Never Trust the Client

In Lab 5, our `POST` validation was minimal: we only checked `isset($input['title'])`. What if the user sends an empty string `""`? What if they send a number or an array instead of a string? What if they send a title that is 5,000 characters long, crashing the database?

A robust API validates all inputs thoroughly and collects *all* validation errors into an array before returning a `422 Unprocessable Entity` (or `400 Bad Request`) response. This allows the client developer to see everything they did wrong in a single request.

Exercise 3: Building a Validation Layer

Step 3A: Create a Validation Function

Add the following validation helper function at the top of your `api.php` file (near your `sendError` function):

```php
function validateBookInput(?array $input): array {
    $errors = [];

    // 1. Check if JSON decoding failed completely
    if ($input === null) {
        sendError(400, "Malformed JSON payload.");
    }

    // 2. Validate 'title'
    if (!isset($input['title']) || !is_string($input['title'])) {
        $errors['title'] = "Title is required and must be a string.";
    } else {
        $title = trim($input['title']);
        if (strlen($title) < 2 || strlen($title) > 255) {
            $errors['title'] = "Title must be between 2 and 255 characters.";
        }
    }

    // 3. Validate 'author'
    if (!isset($input['author']) || !is_string($input['author'])) {
        $errors['author'] = "Author is required and must be a string.";
    } else {
        $author = trim($input['author']);
        if (strlen($author) < 2 || strlen($author) > 255) {
            $errors['author'] = "Author must be between 2 and 255 characters.";
        }
    }

    // 4. Validate optional 'is_available' if provided
    if (isset($input['is_available']) && !is_bool($input['is_available'])) {
        $errors['is_available'] = "is_available must be a boolean (true or false).";
    }

    return $errors;
```

```
}
```

Step 3B: Apply Validation to POST and PUT

Update your `POST` handler in `api.php` to use this new validation function:

```
} elseif ($method === 'POST') {
    $rawJson = file_get_contents('php://input');
    $input = json_decode($rawJson, true);

    // Run validation
    $validationErrors = validateBookInput($input);
    if (!empty($validationErrors)) {
        // 422 Unprocessable Entity is standard for validation failures
        sendError(422, "Validation Failed", $validationErrors);
    }

    // Sanitize by trimming whitespace
    $title = trim($input['title']);
    $author = trim($input['author']);
    $isAvailable = $input['is_available'] ?? true;

    $sql = "INSERT INTO books (title, author, is_available)
            VALUES (:title, :author, :is_available)";
    $stmt = $pdo->prepare($sql);
    $stmt->execute([
        'title' => $title,
        'author' => $author,
        'is_available' => (int)$isAvailable
    ]);

    http_response_code(201);
    echo json_encode([
        "message" => "Book created successfully",
        "id" => (int)$pdo->lastInsertId()
    ]);
}
```

Step 3C: Your Turn - Implement Validated PUT

Locate the `PUT /api.php/books/{id}` block in your regex routing section (from Lab 5, Part 3). Implement the full replacement logic:

1. Read and decode the JSON body.
2. Call `validateBookInput($input)` and return a `422` error using `sendError()` if any errors exist.
3. First, check if the book with `$bookId` actually exists in the database (run a `SELECT id FROM books WHERE id = :id`). If it doesn't exist, call `sendError(404, "Book not found")`.
4. Prepare and execute an `UPDATE books SET title = :title, author = :author, is_available = :avail WHERE id = :id` statement.
5. Return a `200 OK` response with the updated book data.

6. **Test in Postman:** Try sending invalid data (e.g., an empty title `""` and a string `"yes"` for `is_available`) to both `POST` and `PUT`. Verify you get a clean `422` response listing both errors simultaneously!

4 Part 4: Team Project Application

Exercise 4: Upgrading Your Team's Repository

The concepts covered in Labs 5 and 6 form the core grading rubric for your Team Project's backend implementation. Sit down with your team and apply these patterns to your project repository:

1. **Update your OpenAPI Spec (`openapi.yaml`):**
    - Ensure your collection `GET` endpoints document query parameters for `page`, `limit`, and at least two resource-specific filters.
    - Update the `200` response schema for collections to reflect the new `metadata` and `data` envelope structure.
    - Document the `400`, `404`, `406`, `415`, and `422` error responses in your spec.
2. **Implement the Architecture:**
    - Set up your project's Front Controller and database connection.
    - Implement the Content Negotiation and Global Exception Handler middleware so all team members benefit from standardized error reporting.
    - Each team member should take responsibility for one core resource (from your GitHub Kanban board) and implement its paginated `GET` and validated `POST`/`PUT` endpoints.

References and Further Reading

- MDN Web Docs: Content Negotiation: A thorough explanation of how the `Accept` and `Content-Type` headers allow clients and servers to agree on data representations.
- RFC 9457: Problem Details for HTTP APIs: The industry standard specification for formatting JSON error responses in RESTful web services.
- Slim Framework Documentation: Middleware: (For future reference) A great visual guide on how concentric middleware layers wrap around a core application in professional PHP frameworks.
- JSON:API Specification - Pagination: Best practices for structuring pagination links and metadata envelopes in REST responses.
- PHP Manual: Prepared Statements and Stored Procedures: Review how `bindValue()` with `PDO::PARAM_INT` is required when binding `LIMIT` and `OFFSET` clauses in MariaDB/MySQL.